Meta RL

目录

1 Fundamentals of Meta Learning **1**
1.1 MAML 1

2 Data Manipulation **1**
2.1 MentorNet for ReWeighting 1
2.2 Learning Data Manipulation for Augmentation and Weighting 2

3 Task rankings **3**
3.1 PAML 3

4 Eigen-Reptile and ISPL **3**
4.1 Eigen-Reptile Algorithm 3
4.2 ISPL 3

5 Unsupervised Curricula for Visual Meta-Reinforcement Learning **4**

6 Learning Loss for Active Learning **5**

7 BatchBALD **5**

I Fundamentals of Meta Learning

i. MAML

The key idea underlying MAML is to train the model's initial parameters such that the model can reach maximal performance only after a few training iterations.

II Data Manipulation

i. MentorNet for ReWeighting

MentorNet learns a data-driven curriculum dynamically with StudentNet.

Preliminary on Curriculum Learning

Objective: $\min_{\mathbf{w}\in\mathbb{R}^d,\mathbf{v}\sim[0,1]^{n\times m}} \mathbb{F}(\mathbf{w},\mathbf{v}) = \frac{1}{n}\sum_{i=1}^{n} \mathbf{v}_i^T \mathbf{L}(y_i, g_s(x_i,\mathbf{w})) + G(\mathbf{v};\lambda) + \theta||\mathbf{w}||_2^2$.
Self-paced: $G(\mathbf{v}) = -\lambda||\mathbf{v}||_1$.
As shown, the function G specifies a curriculum, i.e., a sequence of samples with their corresponding weights to be used in training.

Algorithm 2 MAML for Few-Shot Supervised Learning

Require: $p(\mathcal{T})$: distribution over tasks
Require: α, β: step size hyperparameters
randomly initialize θ
while not done **do**
  Sample batch of tasks $\mathcal{T}_i \sim p(\mathcal{T})$
  for all $\mathcal{T}_i$ **do**
    Sample K datapoints $\mathcal{D} = \{\mathbf{x}^{(j)}, \mathbf{y}^{(j)}\}$ from $\mathcal{T}_i$
    Evaluate $\nabla_\theta \mathcal{L}_{\mathcal{T}_i}(f_\theta)$ using $\mathcal{D}$ and $\mathcal{L}_{\mathcal{T}_i}$ in Equation (2) or (3)
    Compute adapted parameters with gradient descent: $\theta_i' = \theta - \alpha \nabla_\theta \mathcal{L}_{\mathcal{T}_i}(f_\theta)$
    Sample datapoints $\mathcal{D}_i' = \{\mathbf{x}^{(j)}, \mathbf{y}^{(j)}\}$ from $\mathcal{T}_i$ for the meta-update
  end for
  Update $\theta \leftarrow \theta - \beta \nabla_\theta \sum_{\mathcal{T}_i \sim p(\mathcal{T})} \mathcal{L}_{\mathcal{T}_i}(f_{\theta_i'})$ using each $\mathcal{D}_i'$ and $\mathcal{L}_{\mathcal{T}_i}$ in Equation 2 or 3
end while

Algorithm 3 MAML for Reinforcement Learning

Require: $p(\mathcal{T})$: distribution over tasks
Require: α, β: step size hyperparameters
randomly initialize θ
while not done **do**
  Sample batch of tasks $\mathcal{T}_i \sim p(\mathcal{T})$
  for all $\mathcal{T}_i$ **do**
    Sample K trajectories $\mathcal{D} = \{(\mathbf{x}_1, \mathbf{a}_1, ...\mathbf{x}_H)\}$ using f_θ in $\mathcal{T}_i$
    Evaluate $\nabla_\theta \mathcal{L}_{\mathcal{T}_i}(f_\theta)$ using $\mathcal{D}$ and $\mathcal{L}_{\mathcal{T}_i}$ in Equation 4
    Compute adapted parameters with gradient descent: $\theta_i' = \theta - \alpha \nabla_\theta \mathcal{L}_{\mathcal{T}_i}(f_\theta)$
    Sample trajectories $\mathcal{D}_i' = \{(\mathbf{x}_1, \mathbf{a}_1, ...\mathbf{x}_H)\}$ using $f_{\theta_i'}$ in $\mathcal{T}_i$
  end for
  Update $\theta \leftarrow \theta - \beta \nabla_\theta \sum_{\mathcal{T}_i \sim p(\mathcal{T})} \mathcal{L}_{\mathcal{T}_i}(f_{\theta_i'})$ using each $\mathcal{D}_i'$ and $\mathcal{L}_{\mathcal{T}_i}$ in Equation 4
end while

Learning to approximate predefined curriculums.
Objective: $arg \min_{\Theta} \sum_{(x_i, y_i) \in \mathcal{D}} g_m(\mathbf{z}_i; \Theta) l_i + G(g_m(\mathbf{z}_i; \Theta); \lambda)$.
Learning data-driven curriculums.
Objective: Θ is learned on another dataset $\mathcal{D}' = \{(\phi(\mathbf{x}_i, y_i, \mathbf{w}), v_i^*)\}$, where $(\mathbf{x}_i, y_i)$ is sampled from $\mathcal{D}$ and $|\mathcal{D}'| << |\mathcal{D}|$.

ii. Learning Data Manipulation for Augmentation and Weighting

Equivalence between Data and Reward. The variational policy optimization objective:

$$\mathcal{L}(q, \theta) = \mathbb{E}_{q(x,y)}[R(\mathbf{x}, y|\mathcal{D})] - \alpha KL(q(\mathbf{x}, y)||p(\mathbf{x})p_\theta(y|\mathbf{x})) + \beta H(q) \tag{1}$$

The EM procedure that optimizes q and θ alternatively is:

$$E: q'(\mathbf{x}, y) = \exp\{\frac{\alpha \log p(\mathbf{x}) p_\theta(y|\mathbf{x}) + R(\mathbf{x}, y|\mathcal{D})}{\alpha + \beta}\}/Z$$
$$M: \theta' = arg \max_\theta \mathbb{E}_{q'(\mathbf{x},y)}[\log p_\theta(y|\mathbf{x})]$$

Define the reward function as $R_\delta(\mathbf{x}, y|\mathcal{D}) = \begin{cases} 1 & \text{if}(\mathbf{x}, y) \in \mathcal{D} \\ -\infty & \text{otherwise.} \end{cases}$, and set $\alpha = 0, \beta = 1$, then the policy optimization framework will reduce to supervised learning.
Parameterizing Data Manipulation. Use a relaxed reward $R_\phi(\mathbf{x}, y|\mathcal{D})$ with parameters ϕ.
Learning Manipulation Parameters. After replacing R_δ with R_ϕ, the augmentented M-step can be obtained: $\theta' = arg \max_\theta \mathbb{E}_{exp\{R_\phi(\mathbf{x}, y|\mathcal{D})\}/Z}[\log p_\theta(y|\mathbf{x})]$. Then since θ' is a function of ϕ, thus we can update ϕ':

$$\phi' = arg \max_\phi \mathbb{E}_{(x,y) \sim \mathcal{D}^v}[\log p_{\theta'}(y|\mathbf{x})] \tag{2}$$

Fine-tuning Text Augmentation. $R_\phi^{aug}(\mathbf{x}, y|\mathcal{D}) = \begin{cases} 1 & \text{if } \mathbf{x} \sim g_\phi(\mathbf{x}|\mathbf{x}^*, y),\ (\mathbf{x}^*, y) \in \mathcal{D} \\ -\infty & \text{otherwise.} \end{cases}$

Learning Data Weights. $R_\phi^w(\mathbf{x}, y|\mathcal{D}) = \begin{cases} \phi_i & \text{if } (\mathbf{x}, y) = (\mathbf{x}_i^*, y_i^*), (x_i^*, y_i^*) \in \mathcal{D} \\ -\infty & \text{otherwise} \end{cases}$

III Task rankings

i. PAML

The key idea is to use probabilistic latent task embeddings, then to score potential tasks to learn the next one in latent space.

Probabilistic Meta-Learning. Maximize the ELBO of marginal likelihood $p_\theta(\mathbf{Y}|\mathbf{X})$, then obtain the distribution $p_\theta(\mathbf{Y}|\mathbf{H}, \mathbf{X})$ and $p_\theta(\mathbf{H}|\mathbf{Y}, \mathbf{X}) \approx q_\phi(\mathbf{H}) = \prod_{i=1}^{N} q_\phi(\mathbf{h}_i)$.

Extending the Meta-Learning Model. Maximize the ELBO of marginal likelihood $p_\theta(\mathbf{Y}, \Psi|\mathbf{X})$, in which a new model $p_\theta(\psi_i|\mathbf{h}_i)$ is introduced.

Ranking Candidates in Laten Space. We define the utility of candidate h_* as the self-information/surprisal associated with $\mathbf{h}_*$, under this distribution: $u(\mathbf{h}_*) := -\log \sum_{i=1}^{N} q_{\phi_i}(\mathbf{h}_*) + \log N$.

IV Eigen-Reptile and ISPL

N-way K-shot: K examples for each of the N classes. The objective:

$$\min L(D_{test}, \tilde{\phi}) = -\frac{1}{N} \sum \mathbb{E}\Big[\frac{1}{K} \sum_{(x,y)\in\mathcal{D}_{test}} \log q(\hat{y} = y|x, \phi, \tilde{\phi})\Big], \tilde{\phi} = U^n(D_{train}, \phi) \tag{3}$$

$U^n(D_{train}, \phi)$ means update ϕ for n times, and $q(\hat{y} = y|x, \phi, \tilde{\phi})$ is the predictive distribution.

i. Eigen-Reptile Algorithm

Represent the parameter points by a straight line of the form $\mathbf{w} = \overline{\mathbf{w}} + l\mathbf{e}$, then we get the loss function $J(l_1, l_2, \cdots, l_n, e) = \sum_{i=1}^{n} ||\overline{\mathbf{w}} + l_i\mathbf{e} - \mathbf{w}_i||^2$. The replace l_i with $\mathbf{e}^T(\mathbf{w}_i - \overline{\mathbf{w}})^2$, we get:

$$J(\mathbf{e}) = -\mathbf{e}^T\mathbf{S}\mathbf{e} + \sum_{i=1}^{n} ||\mathbf{w}_i - \overline{\mathbf{w}}||^2 \tag{4}$$

where $\mathbf{S} = \sum_{i=1}^{n}(\mathbf{w}_i - \overline{\mathbf{w}})(\mathbf{w}_i - \overline{\mathbf{w}})^T$ is a scatter matrix. After some calculation, we can find that $\mathbf{e}$ is the eigenvector of $\mathbf{S}$ corresponding to the largest eigenvalue λ. Moreover, we project parameter update vectors $\mathbf{w}_{i+1} - \mathbf{w}_i, i = 1, 2, \cdots, n-1$ on $\mathbf{e}$ to get the corresponding update stepsize ν, so meta-parameters ϕ can be updated as $\phi \leftarrow \phi + \beta\nu\mathbf{e}$.

ii. ISPL

The task-specific loss: $L_{ISPL}(\phi, \mathbf{v}) = \sum_{i=1}^{h} v_i L(x_i, y_i, \phi)$, where $v_i = arg\min_{\mathbf{v}} \frac{v_i}{Q} \sum_{j=1}^{Q} L_j(x_i, y_i, \phi_j) - \gamma v_i$. The key idea is to use emsemble learning to find the noisy labels.

V Unsupervised Curricula for Visual Meta-Reinforcement Learning

The goal is to seek a procedure through which an agent in an environment with visual observations can automatically acquire useful behaviors, as well as how and when to apply them, in effect allowing for *unsupervised* pre-training in visual environments.

We begin with maximizing the mutual information between state trajectories $\tau = (\mathbf{s}_1, \cdots, \mathbf{s}_T)$ and a latent task variable $\mathbf{z}$: $I(\tau; \mathbf{z}) = H(\tau) - H(\tau|\mathbf{z})$.

Algorithm 1: CARML – Curricula for Automatic Reinforcement of Meta-Learning

Require: $\mathcal{C}$, an MDP without a reward function
Initialize f_θ, an RL algorithm parameterized by θ.
Initialize $\mathcal{D}$, a reservoir of state trajectories, via a randomly initialized policy.
while not done **do**
  Fit a task-scaffold q_ϕ to $\mathcal{D}$, e.g. by using Algorithm 2. **E-step §3.2**
  for a desired mixture model-fitting period **do**
    Sample a latent task variable $\mathbf{z} \sim q_\phi(\mathbf{z})$.
    Define the reward function $r_\mathbf{z}(\mathbf{s})$, e.g. by Eq. 8, and a task $\mathcal{T} = \mathcal{C} \cup r_\mathbf{z}(\mathbf{s})$.
    Apply f_θ on task $\mathcal{T}$ to obtain a policy $\pi_\theta(\mathbf{a}|\mathbf{s}, \mathcal{D}_\mathcal{T})$ and trajectories $\{\boldsymbol{\tau}_i\}$.
    Update f_θ via a meta-RL algorithm, e.g. RL^2 [13]. **M-step §3.3**
    Add the new trajectories to the reservoir: $\mathcal{D} \leftarrow \mathcal{D} \cup \{\boldsymbol{\tau}_i\}$.
Return: a meta-learned RL algorithm f_θ tailored to $\mathcal{C}$

E-Step: Task Acquisition. Fit q_ϕ to a reservoir of trajectories produced by f_θ, in turn, q_ϕ gives rise of task distribution $p(\mathcal{T})$. $q_\phi(\mathbf{z})$ will lower-bound the objective:

$$I(\tau; \mathbf{z}) = -\sum_\tau \pi_\theta(\tau) \log \pi_\theta(\tau) + \sum_{\tau, \mathbf{z}} \pi_\theta(\tau, \mathbf{z}) \log \pi_\theta(\tau|\mathbf{z}) \tag{5}$$

$$\geq -\sum_\tau \pi_\theta(\tau) \log \pi_\theta(\tau) + \sum_{\tau, \mathbf{z}} \pi_\theta(\tau|\mathbf{z}) q_\phi(\mathbf{z}) \log q_\phi(\tau|\mathbf{z}) \tag{6}$$

The E-step corresponds to optimizing Eq.6 with respect to ϕ, and thus amounts to fitting q_ϕ to a reservoir of trajectories $\mathcal{D}$ produced by π_θ: $\max_\phi \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}), \tau \sim \mathcal{D}}[\log q_\phi(\tau|\mathbf{z})]$.

M-step: Meta-Learning To allow for state-conditioned Markovian rewards rather than non-Markovian trajectory-level rewards, we lower-bound the trajectory-level MI objective:

$$I(\tau; \mathbf{z}) = \frac{1}{T} \sum_{i=1}^{T} H(\mathbf{z}) - H(\mathbf{z}|\mathbf{s}_1, \cdots, \mathbf{s}_T) \geq \frac{1}{T} \sum_{t=1}^{T} H(\mathbf{z}) - H(\mathbf{z}|\mathbf{s}_t) \tag{7}$$

$$\geq \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}), \mathbf{s} \sim \pi_\theta(\mathbf{s}|\mathbf{z})}[\log q_\phi(\mathbf{s}|\mathbf{z}) - \log \pi_\theta(\mathbf{s})] \tag{8}$$

We make the simplifying assumption that the fitted variaitonal marginal distribution matches that of the policy:

$$\max_\theta \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}), \mathbf{s} \sim \pi_\theta(\mathbf{s}|\mathbf{z})}[\log q_\phi(\mathbf{s}|\mathbf{z}) - \log q_\phi(\mathbf{s})] \tag{9}$$

$$= \max_\theta I(\pi_\theta(\mathbf{s}); q_\phi(\mathbf{z})) - D_{KL}(\pi_\theta(\mathbf{s}|\mathbf{z})) + D_{KL}(\pi_\theta(\mathbf{s}) || q_\phi(\mathbf{s})) \tag{10}$$

Thus optimizing the above objective amounts to maximizing the reward of $r_\mathbf{z}(\mathbf{s}) = \log q_\phi(\mathbf{s}|\mathbf{z}) - \log q_\phi(\mathbf{s})$.

We can also trade-off between component-matching and exploration by introducing a weighting term $\lambda \in [0, 1]$ into $r_{\mathbf{z}}(\mathbf{s})$:

$$r_{\mathbf{z}}(\mathbf{s}) = \lambda \log q_\phi(\mathbf{s}|\mathbf{z}) - \log q_\phi(\mathbf{s}) \tag{11}$$

$$(\lambda - 1) \log q_\phi(\mathbf{s}|\mathbf{z}) + \log q_\phi(\mathbf{z}|\mathbf{s}) + C \tag{12}$$

After having this specific reward function, we can have a specific task incorporating this reward function into our predefined CMP. Then using RL^2 algorithm, we can meta-learn f_θ with these meta-tasks.